Universidade do Estado do Rio de Janeiro - UERJ
Instituto Politécnico - IPRJ

Trabalho Computacional

Modelagem Computacional

Autor:
João Henrique M. P. Gomes-202110048211
Júlia do Amaral Rangel-202310072211
Theo Troncoso Liron-202410086211

Professor:
Prof. Rafael Libotte

Nova Friburgo
14 de abril de 2026

Sumário

1 Introdução **2**

2 Modelagem Matemática **2**
2.1 Determinação dos Parâmetros do Sistema (m, k, c) 2

3 Metodologia **3**
3.1 Solução do Sistema Linear 3
3.2 Cálculo Numérico das Raízes 5
3.2.1 Método da Bisseção e o Método de Newton-Raphson. 5
3.3 Análise Comparativa 10

4 Resultados **10**
4.1 Parâmetros do Sistema Físico 10
4.2 Resposta Temporal do Sistema 11
4.3 Comparação dos Métodos de Busca de Raízes 11
4.3.1 Análise Comparativa dos Métodos 11

5 Conclusão **12**

1 Introdução

A modelagem computacional é uma ferramenta fundamental na engenharia e nas ciências exatas, permitindo a simulação e a análise de sistemas físicos complexos. Dentre estes, o sistema massa-mola-amortecedor se destaca como um modelo clássico para descrever fenômenos oscilatórios amortecidos, encontrados em uma vasta gama de aplicações, desde suspensões automotivas até a análise de vibrações em estruturas civis.

O presente trabalho tem como objetivo central a análise computacional detalhada de um sistema massa-mola-amortecedor linear subamortecido. A análise percorre uma jornada completa, iniciando pela determinação dos parâmetros físicos fundamentais do sistema e culminando na aplicação de métodos numéricos para o estudo de sua resposta dinâmica.

Para atingir esse objetivo, o estudo é estruturado em etapas sequenciais. Primeiramente, os parâmetros essenciais do sistema, como massa (m), constante elástica (k) e coeficiente de amortecimento (c) são determinados a partir de um sistema de equações lineares fornecido, utilizando-se para isso o método de Eliminação de Gauss.

Uma vez definidos os parâmetros, a solução da equação diferencial que descreve a posição da massa em função do tempo é utilizada para gerar uma representação gráfica do movimento. Esta análise visual, é crucial, pois fornece a base intuitiva para a etapa subsequente: a determinação dos instantes de tempo em que a massa cruza a posição de repouso, ou seja, as raízes da função de posição.

Para o cálculo das três primeiras raízes positivas, dois métodos numéricos clássicos de busca de raízes são implementados: o Método da Bisseção e o Método de Newton-Raphson. As estimativas iniciais necessárias para ambos os métodos são obtidas diretamente da inspeção do gráfico de posição.

Finalmente, o trabalho apresenta uma análise comparativa do desempenho dos dois algoritmos de busca de raízes. Esta comparação avalia a eficiência de cada método com base no número de iterações necessárias e na precisão alcançada, tendo como critério de parada a obtenção de uma raiz x_r tal que $|y(x_r)| < 10^{-5}$, onde y representa a função de posição.

2 Modelagem Matemática

2.1 Determinação dos Parâmetros do Sistema (m, k, c)

A primeira etapa do trabalho consiste em encontrar os valores dos parâmetros físicos do sistema: a massa (m), a constante elástica (k) e o coeficiente de amortecimento (c). Foi fornecido um sistema de 3 equações lineares com 3 incógnitas.

$$\begin{cases} \frac{1}{3}m + c + 2k &= \frac{403}{98} \\ 2m + \frac{1}{2}c + k &= \frac{909}{196} \\ m - \frac{1}{3}c + 2k &= \frac{1289}{294} \end{cases} \tag{1}$$

A implementação computacional do Método de Gauss será realizada, permitindo obter os valores numéricos de m, k e c que serão utilizados em todas as etapas subsequentes.

De posse dos parâmetros do sistema, a segunda etapa foca na compreensão do seu comportamento dinâmico. A equação diferencial que descreve o sistema é dada por:

$$my'' + cy' + ky = 0 \tag{2}$$

Com a condição de amortecimento subamortecido $(c^2 < 4mk)$, a solução analítica desta EDO $y(t)$,

$$y(t) = e^{\alpha t}\left(\cos(\beta t) - \frac{\alpha}{\beta}\sin(\beta t)\right) \tag{3}$$

3 Metodologia

Este trabalho foi desenvolvido utilizando uma abordagem híbrida. As simulações numéricas e a implementação dos algoritmos foram realizadas na linguagem C++ para otimizar o desempenho computacional. A análise de dados, a geração de gráficos e a prototipagem inicial foram conduzidas em Python 3, com o auxílio das bibliotecas Pandas, NumPy e Matplotlib, em um ambiente Jupyter Notebook.

3.1 Solução do Sistema Linear

Para solucionar o sistema linear apresentado na Eq.(1), optou-se pelo método da Eliminação de Gauss. O método da Eliminação de Gauss consiste em transformar a matriz aumentada em uma matriz triangular superior por meio de operações elementares de linha. Isso permite que a solução seja encontrada por retro-substituição, resolvendo primeiro a última incógnita e, em seguida, substituindo os valores encontrados para determinar as incógnitas restantes. Optou-se por implementar a matriz diretamente no código, representando os coeficientes em sua forma fracionária. Esta escolha evitou a perda de precisão inerente às representações decimais inviáveis, deixando as aproximações a cargo do compilador.

Código em C++:

```
#include <iostream>
int main() {


   double matriz[3][4] = {
        {1.0/3.0, 1.0, 2.0, 403.0/98.0},
        {2.0, 1.0/2.0, 1.0, 909.0/196.0},
        {1.0, -1.0/3.0, 2.0, 1289.0/294.0}
    };

    //Agora vamos escrever o algoritmo de Eliminacao Gaussiana

    for (int j=0; j <3-1; j++){


        for (int i=j+1; i <3; i++){

```

```
            // Verificacao do Pivo e Calculo do Multiplicador
            if (matriz[j][j] == 0) {
                std::cout << "Erro: Pivo zero encontrado." << std
                    ::endl;
                return 1;
            }

            // Calculo do Multiplicador das linhas
            double d = matriz[i][j] / matriz[j][j];

            // Laco para realizar as operacoes com linhas
            for (int k = j; k < 4; k++) {

                matriz[i][k] = matriz[i][k] - d * matriz[j][k];
            }
        }
    }


    for (int i=0; i<3; i++){
        for(int j=0;j<4;j++){
            std::cout<<matriz[i][j] <<" ";
        }
        std::cout<<std::endl;
    }

    //Criamos as variasveis para receber os valores solucao do
        sistema

    double a, b, c;
    c = matriz[2][3] / matriz[2][2];
    b = (matriz[1][3] - matriz[1][2] * c) / matriz[1][1];
    a = (matriz[0][3] - matriz[0][2] * c - matriz[0][1] * b) /
        matriz[0][0];


    std::cout << "a = " << a << std::endl;
    std::cout << "b = " << b << std::endl;
    std::cout << "c = " << c << std::endl;


    return 0;
}
```

Listing 1: Solução do Sistema Linear

Utilizando a biblioteca Matplotlib da linguagem Python, esta função y(t) foi plotada em um intervalo de tempo $t[0, 10]$. O objetivo desta visualização é duplo: primeiramente, validar visualmente o comportamento oscilatório amortecido esperado; em segundo lugar, identificar graficamente as regiões aproximadas onde ocorrem as três primeiras raízes positivas da função, ou seja, os instantes $t > 0$ em que $y(t) = 0$.

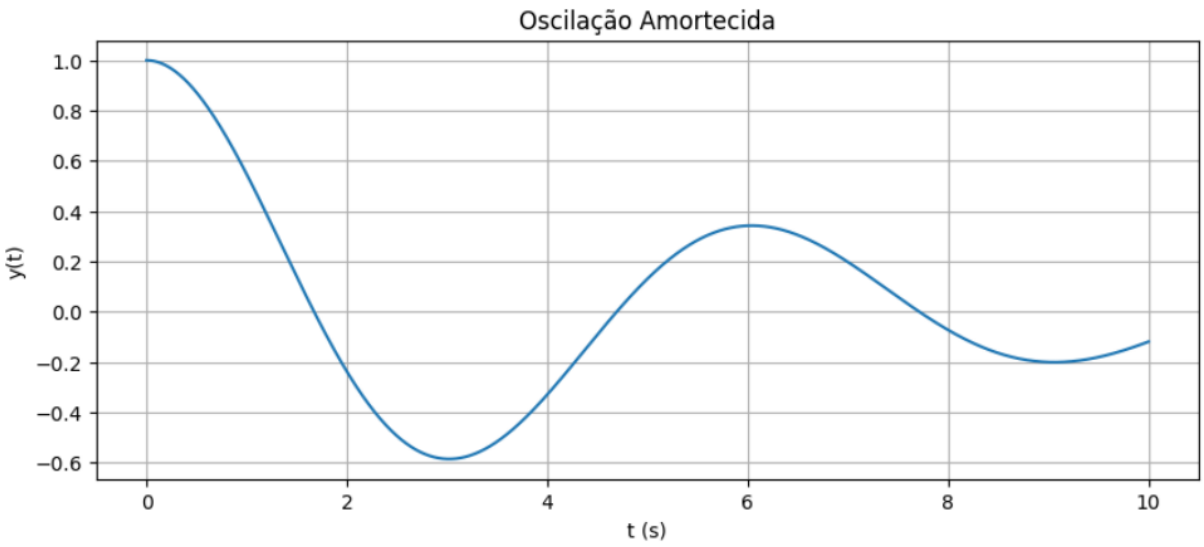

Figura 1: Amplitude do sistema massa-mola-amortecedor em função tempo.

3.2 Cálculo Numérico das Raízes

Caminhando para o fim do trabalho, a tarefa consiste em encontrar, com alta precisão, os valores das três primeiras raízes positivas de y(t), que representam os instantes em que a massa passa pelo ponto de repouso. Para isso, serão aplicados dois métodos numéricos: o Método da Bisseção e o Método de Newton-Raphson. A implementação de ambos os algoritmos será feita em Python 3.

3.2.1 Método da Bisseção e o Método de Newton-Raphson.

```python
from math import exp, cos, sin, sqrt

# PARAMETROS
precisao_percentual = 0.01   # tolerancia de erro relativo (%)
iteracoes_maximas = 200
criterio_fun = 1e-5          # |f(x)| < 10^-5

# CONSTANTES (alpha -> alfa, beta -> beta)
alfa = -49.0 / 276.0
beta = sqrt(82607.0) / 276.0


# FUNCAO Y(t)
def resposta_posicao(t, alfa=alfa, beta=beta):
    return exp(alfa * t) * (cos(beta * t) - ((alfa / beta) * sin(
        beta * t)))


# DERIVADA
def derivada_resposta(t, alfa=alfa, beta=beta):
```

```python
    return (-(alfa**2 + beta**2) / beta) * exp(alfa * t) * sin(beta * t)


# METODO DA BISSECAO
def metodo_bisseccao(func, a, b, tol_percent=0.01, max_iter=1000, tol_fun=1e-5):
    fa = func(a)
    fb = func(b)

    if fa * fb > 0:
        raise ValueError(f"f(a) e f(b) tem o mesmo sinal no intervalo [{a}, {b}].")

    lista_aproximacoes = []
    xn = None
    xn_anterior = None
    erro_percentual = float('inf')

    for it in range(1, max_iter + 1):
        xn = 0.5 * (a + b)
        fxn = func(xn)
        lista_aproximacoes.append(xn)

        # CRITERIO DE PARADA
        if abs(fxn) < tol_fun:
            return {
                "raiz": xn,
                "erro_percentual": erro_percentual,
                "iteracoes": it,
                "historico": lista_aproximacoes,
                "criterio": "abs(f(x)) < tol_fun"
            }

        # atualizacao do intervalo
        if fa * fxn < 0:
            b = xn
            fb = fxn
        else:
            a = xn
            fa = fxn

        # erro relativo
        if xn_anterior is not None:
            if xn != 0:
                erro_percentual = abs((xn - xn_anterior) / xn) * 100.0
            else:
                erro_percentual = abs(xn - xn_anterior)

        xn_anterior = xn
```

```python

        # criterios classicos
        if erro_percentual < tol_percent or abs(b - a) < 1e-12:
            return {
                "raiz": xn,
                "erro_percentual": erro_percentual,
                "iteracoes": it,
                "historico": lista_aproximacoes,
                "criterio": "erro_percentual ou intervalo pequeno"
            }

    raise RuntimeError("Bissecao nao convergiu dentro do limite de iteracoes.")


# METODO NEWTON-RAPHSON
def metodo_newton_raphson(func, dfunc, chute_inicial,
                          tol_percent=0.01, max_iter=200,
                          eps_deriv=1e-14, tol_fun=1e-5):

    historico_valores = []
    historico_erros = []
    x_atual = chute_inicial

    for it in range(1, max_iter + 1):
        fx = func(x_atual)
        fpx = dfunc(x_atual)

        # CRITERIO DE PARADA
        if abs(fx) < tol_fun:
            return {
                "raiz": x_atual,
                "erro_percentual": 0.0,
                "iteracoes": it,
                "historico": historico_valores,
                "erros": historico_erros,
                "criterio": "abs(f(x)) < tol_fun"
            }

        if abs(fpx) < eps_deriv:
            raise ZeroDivisionError(f"Derivada muito pequena em x={x_atual}")

        x_proximo = x_atual - fx / fpx
        historico_valores.append(x_proximo)

        # erro relativo
        if x_proximo != 0:
            erro_percentual = abs((x_proximo - x_atual) / x_proximo) * 100.0
```

```python
        else:
            erro_percentual = abs(x_proximo - x_atual)

        historico_erros.append(erro_percentual)

        if erro_percentual < tol_percent:
            return {
                "raiz": x_proximo,
                "erro_percentual": erro_percentual,
                "iteracoes": it,
                "historico": historico_valores,
                "erros": historico_erros,
                "criterio": "erro_percentual"
            }

        x_atual = x_proximo

    raise RuntimeError("Newton-Raphson nao convergiu dentro do limite.")


# ENCONTRAR RAIZES PELA BISSECAO
intervalos_para_bisseccao = [(1.5, 2.0), (4.0, 5.0), (7.0, 8.0)]
resultados_bisseccao = []

print("RESULTADOS: METODO DA BISSECAO")
for i, (ai, bi) in enumerate(intervalos_para_bisseccao, start=1):
    try:
        res_b = metodo_bisseccao(resposta_posicao, ai, bi,
                                 tol_percent=precisao_percentual,
                                 max_iter=iteracoes_maximas,
                                 tol_fun=criterio_fun)
        resultados_bisseccao.append(res_b)

        print(f"\nRaiz #{i} (bissecao) no intervalo [{ai}, {bi}]:")
        print(f"  raiz = {res_b['raiz']}")
        print(f"  erro percentual = {res_b['erro_percentual']}")
        print(f"  iteracoes = {res_b['iteracoes']}")
        print(f"  criterio de parada: {res_b['criterio']}")
    except Exception as e:
        print(f"Erro no intervalo [{ai}, {bi}]: {e}")
        resultados_bisseccao.append(None)


# NEWTON-RAPHSON
chutes_newton = [1.6, 5.0, 7.5]
resultados_newton = []

print("\n\nRESULTADOS: NEWTON-RAPHSON")
for i, chute in enumerate(chutes_newton, start=1):
```

```python
    try:
        res_n = metodo_newton_raphson(resposta_posicao, derivada_resposta,
                                      chute,
                                      tol_percent=precisao_percentual,
                                      max_iter=iteracoes_maximas,
                                      tol_fun=criterio_fun)
        resultados_newton.append(res_n)

        print(f"\nRaiz #{i} (Newton) a partir de x0 = {chute}:")
        print(f"  raiz = {res_n['raiz']}")
        print(f"  erro percentual = {res_n['erro_percentual']}")
        print(f"  iteracoes = {res_n['iteracoes']}")
        print(f"  criterio de parada: {res_n['criterio']}")

    except Exception as e:
        print(f"Erro (Newton) a partir de x0={chute}: {e}")
        resultados_newton.append(None)


# COMPARACAO DOS METODOS
def comparar_metodos(resultados_bis, resultados_newt):
    total = max(len(resultados_bis), len(resultados_newt))
    Mnr_Inter = {"bisseccao": 0, "newton": 0, "equivalencia": 0}

    print("\n\n COMPARACAO ENTRE METODOS (RESIDUAL |y(raiz)|)")

    for idx in range(total):
        print(f"\nRaiz #{idx+1}:")
        res_b = resultados_bis[idx]
        res_n = resultados_newt[idx]

        # Bissecao
        if res_b is None:
            print("  Bissecao: sem resultado.")
            residual_b = float('inf')
        else:
            raiz_b = res_b["raiz"]
            residual_b = abs(resposta_posicao(raiz_b))
            print(f"  Bissecao -> raiz {raiz_b}, residual |y| = {residual_b}")

        # Newton
        if res_n is None:
            print("  Newton: sem resultado.")
            residual_n = float('inf')
        else:
            raiz_n = res_n["raiz"]
            residual_n = abs(resposta_posicao(raiz_n))
```

```python
            print(f"  Newton -> raiz {raiz_n}, residual |y| = {residual_n}")

        # Metodo mais eficiente (iteracoes)
        if residual_b < residual_n:
            print("  Menor numero de iteracoes: Bissecao")
            Mnr_Inter["bisseccao"] += 1
        elif residual_n < residual_b:
            print("  Menor numero de iteracoes: Newton")
            Mnr_Inter["newton"] += 1
        else:
            print("  Equivalencia")
            Mnr_Inter["equivalencia"] += 1

    print("\nResumo geral:")
    print(f"  Bissecao teve o menor numero de iteracoes: {Mnr_Inter['bisseccao']} vezes")
    print(f"  Newton teve o menor numero de iteracoes:   {Mnr_Inter['newton']} vezes")
    print(f"  Mnr_Inter:         {Mnr_Inter['equivalencia']}")

comparar_metodos(resultados_bisseccao, resultados_newton)
```

Listing 2: Calculando Raizes da função e Comparando Métodos

3.3 Análise Comparativa

Após a obtenção das raízes por ambos os métodos, uma comparação de desempenho será realizada, focando no número de iterações necessárias para que cada método alcance a precisão estipulada.

4 Resultados

Esta seção apresenta os resultados numéricos obtidos ao aplicar a metodologia descrita anteriormente.

4.1 Parâmetros do Sistema Físico

A solução do sistema de equações lineares utilizando o Método de Eliminação de Gauss forneceu os parâmetros físicos do sistema massa-mola-amortecedor, conforme apresentado na Tabela 1.

Tabela 1: Parâmetros do sistema obtidos.

Parâmetro	Símbolo	Valor Obtido
Massa	m	[1.40816]
Constante Elástica	k	[1.57143]
Coef. Amortecimento	c	[0.5]

Com estes valores, pode-se verificar a condição de subamortecimento:

$$c^2 = [0.25] < 4mk = [8.8512]$$

Confirmando o comportamento oscilatório amortecido esperado, cuja equação de posição $y(t)$ é dada pela Eq.3.

4.2 Resposta Temporal do Sistema

A Figura 2 ilustra a resposta temporal do sistema, $y(t)$, plotada para um intervalo de [0, 10] segundos.

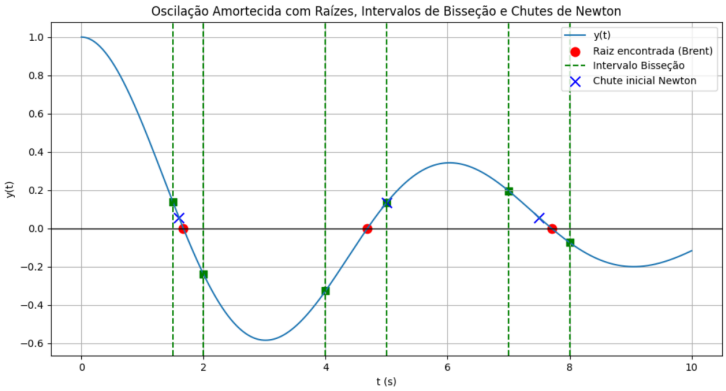

Figura 2: Posição da massa $y(t)$ em função do tempo, demonstrando o comportamento oscilatório subamortecido. Os círculos vermelhos indicam as três primeiras raízes positivas.

Como observado na Figura 2, o sistema oscila em torno da posição de equilíbrio ($y = 0$) com amplitude decrescente. A partir desta visualização foi fundamental permitindo observar com clareza os intervalos iniciais para o Método da Bisseção e das estimativas iniciais para o Método de Newton-Raphson.

Realizado a inspeção gráfica, os intervalos e chutes iniciais para as três primeiras raízes (r_1, r_2, r_3) foram definidos como:

- **Raiz 1 (r_1):** Intervalo $[1.5, 2]$ e chute inicial $x_{0,1} = 1.6$.
- **Raiz 2 (r_2):** Intervalo $[4, 5]$ e chute inicial $x_{0,2} = 5.0$.
- **Raiz 3 (r_3):** Intervalo $[7, 8]$ e chute inicial $x_{0,3} = 7.5$.

4.3 Comparação dos Métodos de Busca de Raízes

Ambos os métodos, Bisseção e Newton-Raphson, foram implementados para encontrar as três primeiras raízes com o critério de parada de $|y(x_r)| < 10^{-5}$. Os resultados comparativos estão consolidados na Tabela 2.

4.3.1 Análise Comparativa dos Métodos

A Tabela 2 apresenta um comparativo quantitativo entre os métodos da Bisseção e de Newton-Raphson. A análise dos dados permite extrair conclusões importantes sobre a eficiência numérica de cada algoritmo.

Tabela 2: Resultados comparativos dos métodos da Bisseção e Newton-Raphson.

Raiz	Método	Chute / Intervalo Inicial	Raiz Encontrada (x_r)	Iterações (N)	$\lvert y(x_r)\rvert$
r_1	Bisseção	[1.5, 2]	[1.67053]	[12]	[2.952410e-0 ¡ 10e-5]
	Newton-Raphson	x_0 =[1.6]	[1.67056]	[3]	[7.808692e-08 ¡ 10e-5]
r_2	Bisseção	[4, 5]	[4.68725]	[12]	[6.518694e-05 ¡ 10e-5]
	Newton-Raphson	x_0 =[5.0]	[4.68739]	[3]	[2.146769e-09 ¡ 10e-5]
r_3	Bisseção	[7, 8]	[7.70458]	[11]	[9.800332e-05 ¡ 10e-5]
	Newton-Raphson	x_0 =[7.5]	[7.70422]	[3]	[8.454458e-07 ¡ 10e-5]

Eficiência Iterativa e Convergência: É possível notar uma diferença significativa no número de iterações (N) necessárias para atingir o critério de parada. O método de Newton-Raphson demonstrou uma eficiência superior, atingindo a precisão desejada em apenas 3 iterações para todas as raízes encontradas. Em contrapartida, o método da Bisseção exigiu entre 11 e 12 iterações para realizar a mesma tarefa.

Este comportamento enfatiza a teoria numérica, evidenciando a distinção entre a **convergência quadrática** do método de Newton-Raphson (onde o erro diminui drasticamente a cada passo) e a **convergência linear** da Bisseção. Portanto, embora o tempo de processamento não tenha sido medido, o método de Newton exigiu significativamente menos passos computacionais para refinar a solução.

Precisão dos Resultados: Além da redução no número de passos, a precisão final obtida (representada por $|y(x_r)|$) também foi superior no método de Newton-Raphson. Enquanto o método da Bisseção encerrou o processo assim que o erro cruzou o limiar de 10^{-5}, o método de Newton-Raphson atingiu erros residuais na ordem de 10^{-8} e 10^{-9} dentro dessas poucas iterações.

Influência das Condições Iniciais: É importante ressaltar que a eficiência do método de Newton-Raphson foi favorecida pela escolha adequada das estimativas iniciais (x_0), obtidas através da análise gráfica prévia. A proximidade dos chutes iniciais às raízes reais garantiu que o método operasse dentro de sua região de convergência, evitando divergências.

Conclusão da Análise: Conclui-se que, para este sistema, o método de Newton-Raphson apresentou melhor desempenho numérico, alcançando maior precisão com um custo iterativo aproximadamente quatro vezes menor que o da Bisseção (3 iterações contra 12).

5 Conclusão

O presente trabalho apresentou o desenvolvimento e a aplicação de métodos de modelagem computacional para a análise de um sistema físico massa-mola-amortecedor. A abordagem percorreu um ciclo completo de engenharia, partindo da identificação de parâmetros através de sistemas lineares, passando pela visualização do comportamento dinâmico e culminando na determinação precisa das raízes da equação de movimento.

A etapa inicial de resolução do sistema linear (via Método de Gauss) provou-se essencial para definir corretamente as constantes físicas (m, c, k), sem as quais a análise subsequente seria impossível. A visualização gráfica da solução da EDO, por sua vez,

não serviu apenas como ilustração, mas como uma ferramenta essencial para fornecer as estimativas iniciais necessárias aos algoritmos de busca de raízes.

Na análise comparativa dos métodos numéricos, os resultados confirmaram a teoria. Embora o Método da Bisseção tenha se mostrado robusto e capaz de convergir, o Método de Newton-Raphson demonstrou uma eficiência iterativa superior. A convergência quadrática do método de Newton permitiu atingir a precisão desejada e erros residuais mínimos com apenas 3 iterações, contra 11 a 12 iterações da Bisseção.

Conclui-se, portanto, que para sistemas onde a derivada é conhecida e boas estimativas iniciais estão disponíveis, o Método de Newton-Raphson é a escolha preferencial. Este estudo de caso reforça a importância da modelagem computacional não apenas para resolver equações, mas para selecionar as ferramentas numéricas mais eficientes para cada tipo de problema de engenharia.